GREEDY

PRESENTED BY COMPETITIVE PROGRAMMING CLUB

CONCEPT OF GREEDY

What is Greedy?
Problems that can be solved by selecting the choice that seems to be the best at the moment at every step.

Well Greedy is ~~being King Midas and turning everything into gold~~ finding the optimal solution at every step and put those “best” solutions at every step together to create a “best” solution overall.

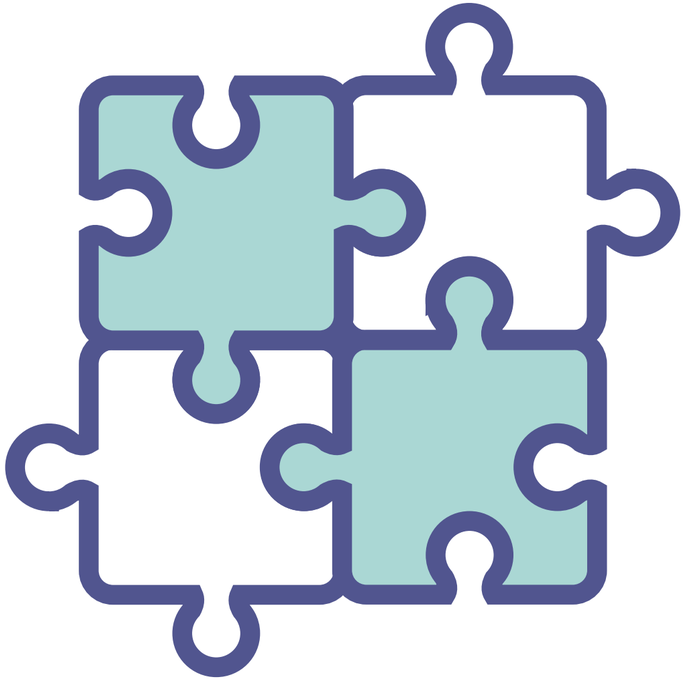
When using greedy, make sure to verify that those best solutions at every step do create a best solution overall.

Greedy isn’t an algorithm, but instead a way of thought and a way of completing problems. Therefore, there isn’t an exact template that you can remember and apply onto every problem, but instead you need to think of how to find the optimum solution at every step and then put them together

What that means

Example: Going through a grid where you cannot go back, you can only go to the upper-right, right, lower-right grid. Maximize the amount of coins picked (each grid has a number of coins)

- Greedy: choose the greatest next step. This doesn't ensure that you maximize the amount of coins unless under certain constraints.
- Non-Greedy/Complete Search: Try all paths and compare results.



Whats Hard?

1. Realizing that it is greedy
2. What ‘best’ is

2020 Jan. Bronze Problem 3 – Race

Bessie is running a race of length K ($1 \leq K \leq 10^9$) meters. She starts running at a speed of 0 meters per second. In a given second, she can either increase her speed by 1 meter per second, keep it unchanged, or decrease it by 1 meter per second. For example, in the first second, she can increase her speed to 1 meter per second and run 1 meter, or keep it at 0 meters per second and run 0 meters. Bessie's speed can never drop below zero.

Bessie will always run toward the finish line, and she wants to finish after an integer amount of seconds (ending either at or past the goal line at this integer point in time). Furthermore, she doesn't want to be running too quickly at the finish line: at the instant in time when Bessie finishes running K meters, she wants the speed she has just been traveling to be no more than X ($1 \leq X \leq 10^5$) meters per second. Bessie wants to know how quickly she can finish the race for N ($1 \leq N \leq 1000$) different values of X.

SCORING:

- Test cases 2-4 satisfy $N = X = 1$.
- Test cases 5-10 satisfy no additional constraints.

INPUT FORMAT (file race.in):

The first line will contain two integers K and N.

The next N lines each contain a single integer X.

OUTPUT FORMAT (file race.out):

Output N lines, each containing a single integer for the minimum time Bessie needs to run K meters so that she finishes with a speed less than or equal to X.

SAMPLE INPUT:

```
10 5
1
2
3
4
5
```

SAMPLE OUTPUT:

```
6
5
5
4
4
```

```python
def solve(d):
    mins=int(input())
    lhstravel=rhstravel=timeused=0
    currspeed=1
    while True:
        lhstravel+=currspeed
        timeused+=1
        if lhstravel+rhstravel>=d:
            return timeused
        if currspeed>mins:
            rhstravel+=currspeed
            timeused+=1
            if lhstravel+rhstravel>=d:
                return timeused
        currspeed+=1

k=input().split()
n=int(k[1])
k=int(k[0])
for i in range(n):
    print(solve(k))
```

What we do here is that instead of thinking “how fast we can go”, think of the maximum distance that can be covered while having an ending speed less than K. We would want her always to be accelerating/decelerating since we would want her to have a maximum speed that would enable her to have the maximum distance.

We would find out that if we are going to have a maximum speed thats greater than speed limit, then we are going to have all the speeds in the middle for 2 seconds. Thus we realize that what we can do is that we can first add up our speed until our speed limit, and then calculate the time used to decrease our speed to the speed limit later on while we are increasing our speed to the maximum.

English (en)

Farmer John's cousin Ben happens to be a mad scientist. Normally, this creates a good bit of friction at family gatherings, but it can occasionally be helpful, especially when Farmer John finds himself facing unique and unusual problems with his cows.

Farmer John is currently facing a unique and unusual problem with his cows. He recently ordered N cows ($1 \leq N \leq 1000$) consisting of two different breeds: Holsteins and Guernseys. He specified the cows in his order in terms of a string of N characters, each either H (for Holstein) or G (for Guernsey). Unfortunately, when the cows arrived at his farm and he lined them up, their breeds formed a different string from this original string.

Let us call these two strings A and B, where A is the string of breed identifiers Farmer John originally wanted, and B is the string he sees when his cows arrive. Rather than simply check if re-arranging the cows in B is sufficient to obtain A, Farmer John asks his cousin Ben to help him solve the problem with his scientific ingenuity.

After several months of work, Ben creates a remarkable machine, the multi-cow-breed-flipinator 3000, that is capable of taking any substring of cows and toggling their breeds: all Hs become Gs and all Gs become Hs in the substring. Farmer John wants to figure out the minimum number of times he needs to apply this machine to transform his current ordering B into his original desired ordering A. Sadly, Ben's mad scientist skills don't extend beyond creating ingenious devices, so you need to help Farmer John solve this computational conundrum.

INPUT FORMAT (file breedflip.in):

The first line of input contains N, and the next two lines contain the strings A and B. Each string has N characters that are either H or G.

OUTPUT FORMAT (file breedflip.out):

Print the minimum number of times the machine needs to be applied to transform B into A.

SAMPLE INPUT:

```
7
GHHHGHH
HHGGGHH
```

SAMPLE OUTPUT:

```
2
```

First, FJ can transform the substring that corresponds to the first character alone, transforming B into GHGGGHH. Next, he can transform the substring consisting of the third and fourth characters, giving A. Of course, there are other combinations of two applications of the machine that also work.

Problem credits: Brian Dean

Intuition

Flip every mismatching substring until the term matches

Intuition	GHHH HHGG	GHHH GHGG	GHHH GHHH	Min: 2 Times	number of mismatching substrings n
Change More	GHHH HHGG	GHHH GGHH	GHHH GHHH	Min: 2 Times	1 + number of matching substrings 1 + n -1 = n

(only consider cases where mismatch-match-mismatch...
Essentially end with mismatch, so number of substrings must be n-1.)

Code

crazy_scientist.py

```python
f = open('breedflip.in','r')
N = int(f.readline())
A = f.readline()
B = f.readline()
f.close()
flag = 0
cnt = 0
for i in range(N):
    if B[i] != A[i]:
        flag = 1
    elif flag:
        cnt+=1
        flag = 0
f = open('breedflip.out','w')
f.write(str(cnt+flag))
f.close()
```

English (en)

Feeling ambitious, Farmer John plans to attempt something that never seems to go quite right: he wants to take a photograph of his entire herd of cows.

To make the photograph look nice, he wants the cows to line up in a single row from shortest to tallest. Unfortunately, right after he has the cows line up this way, Bessie the cow, always the troublemaker, steps out of line and re-inserts herself at some other location in the lineup!

Farmer John would like to swap pairs of cows so the entire herd is again lined up properly. Please help him determine the minimum number of swaps he needs to make between pairs of cows in order to achieve this goal.

INPUT FORMAT (file outofplace.in):

The first line of input contains N $(2 \leq N \leq 100)$. The next N lines describe the heights of the cows as they are lined up after Bessie makes her move. Each cow height is an integer in the range $1 \ldots 1,000,000$. Cows may have the same height.

OUTPUT FORMAT (file outofplace.out):

Please output the minimum number of times Farmer John needs to swap pairs of cows in order to achieve a proper ordering. Swaps do not necessarily need to involve adjacent cows in the ordering.

SAMPLE INPUT:

```
6
2
4
7
7
9
3
```

SAMPLE OUTPUT:

```
3
```

In this example, Bessie is clearly the cow of height 3. FJ return the cows to sorted order using three swaps as described below:

```
2 4 7 7 9 3 - Original Lineup
2 4 7 7 3 9 - Swap the last two cows
2 4 3 7 7 9 - Swap the first 7 and 3
2 3 4 7 7 9 - Swap 4 and 3
```

Problem credits: Brian Dean

Chain of Thought

1. Obviously, we don’t want redundant swaps
2. always exists substring such that bessie is either min at max‘s position or max at min’s position (substring of bessie & where bessie should be)
3. Min swaps are needed to fix that substring?
4. For one swap, can correct one position
5. for K distinct numbers, would require K-1 (last one already sorted after K-1, cause there’s only one position left)
6. Sort() only considers distinct cases, so we can use different digits of sorted array to original array - 1 to generalize this calculation. (This is equivalent to -1 for every repeating digit).

Conclusion

1. Sort the array, compare to see how many digits are different. That number -1 will be the min swaps required. O(n)
2. Find Bessie and its supposed location, count length of substring, output length -1 -1 * number of repeating digits.

outOfOrder.py > ...

```python
f = open('outofplace.in','r')
N = int(f.readline())
h = []
for i in range(N):
    h.append(int(f.readline()))
f.close()
h_copy = h.copy()
h_copy.sort()
cnt = 0
for i in range(N):
    if h[i] != h_copy[i]:
        cnt+=1
f = open('outofplace.out','w')
f.write(str(cnt-1))
f.close()
```

G10 CS H Midterm

Given number N, delete M Digits from it, how to minimize it? (all the preceding 0s will be deleted at the end of computing)

96570 del 2

naive del max → 650

True min → 570

```python
def remove(number,k):
    nums=[]
    for d in number:
        while k>0 and nums and nums[-1]>d:
            nums.pop()
            k-=1
        nums.append(d)
    res=''.join(nums[:len(nums)-k]).lstrip('0')
    if res==='':
        return '0'
    return res

n = input().lstrip('0')
m = int(input())
print(remove(n, m))
```

Thank you very much!

COMPETITIVE PROGRAMMING CLUB